



FlowBoard

Technical & Product Documentation

A real-time collaborative task board — Flutter + Firebase, built end to end as a portfolio-grade engineering project.

Live application: <https://flowboard-app-7539.web.app>

Version documented: 1.0.0 (12 commits, initial scaffold through Apple sign-in)

Stack: Flutter (Web + Android), Riverpod, Firebase (Firestore, Auth, Hosting), Google Gemini

Document date: September 2026

Table of Contents

1	Overview & Purpose
2	Feature Summary
3	Technology Stack
4	System Architecture
5	Data Model (Firestore Schema)
6	Authentication & Identity
7	Security Model & Permissions
8	Real-Time Collaboration Mechanics
9	Feature Deep Dives
10	UI/UX & Design System
11	Project Structure
12	Getting Started (Local Development)
13	Testing Strategy
14	Deployment
15	Known Limitations & Future Work
16	Development History
17	Appendix A — Full Firestore Security Rules
18	Appendix B — Configuration & Environment Variables

1. Overview & Purpose

FlowBoard is a Trello-style collaborative task board built to demonstrate real, non-trivial engineering — not a CRUD demo. Every piece of "real-time" behavior in the app is backed by an actual Firestore listener chain, not a simulated or mocked interaction: dragging a card, another tab seeing it move instantly; two people editing the same card and getting a genuine conflict banner; a teammate's cursor-adjacent presence pill appearing and fading based on a live heartbeat.

The project was built iteratively, starting from a static design reference (a Claude-generated design kit exported as HTML/CSS tokens) and growing into a fully wired Firebase backend: real authentication (Google, Apple, email/password, guest), a genuine per-board membership and role system enforced by Firestore Security Rules (not just hidden UI), AI-assisted task breakdown via the Gemini API, and three independent ways to invite someone to a board (email lookup, username lookup, and a shareable join link).

Design constraint carried through the whole project: no paid infrastructure. Every feature — including security rules enforcement, avatar photo storage, and invite links — was deliberately built to avoid requiring the Firebase Blaze (pay-as-you-go) plan, Cloud Functions, or Firebase Storage (which now requires Blaze even for free-tier usage). Where a paid approach would have been the "obvious" solution (e.g. Cloud Functions for a key-protecting backend proxy, Storage for photos), a free alternative was used instead and the trade-off is documented inline in this doc and in code comments.

The application is deployed and publicly reachable at <https://flowboard-app-7539.web.app>. It also builds for Android (verified via `flutter build apk`); an iOS build was not attempted since that requires macOS/Xcode, unavailable in this project's development environment.

2. Feature Summary

Boards & Tasks

- Create, rename, recolor, archive, restore, and permanently delete boards
- Three-column workflow (To Do / In Progress / Done) with drag-and-drop reordering
- Reordering uses a **fractional index** so moving one card never requires rewriting its siblings
- Tasks carry title, description, priority (Low/Medium/High), assignee, due date, subtasks, comments, and an activity timeline
- Real task deletion (with confirmation)

Real-Time Collaboration

- Live presence: see who else currently has a board open, with automatic staleness cleanup (heartbeat-based, since Firestore has no native disconnect signal)
- Conflict detection: if someone else edits a card while you have it open, a banner appears naming who changed it and when — computed from a real `updatedAt` / `updatedBy` comparison against the value captured when you opened the sheet, not simulated
- Every mutation is a real Firestore write; every client sees it through the same snapshot listener — there is no local-only optimistic state that could drift from the server

AI

- "AI subtasks": calls the Google Gemini API to break a vague task into a concrete checklist
- Falls back to a static suggestion list if no API key is configured, so the feature always demos even without credentials

Identity & Auth

- Sign in with Google, Apple, email/password, or as a guest (anonymous)
- Guest → real account upgrade via Google account linking
- Handles Firebase's ambiguous `invalid-credential` error (which conflates "wrong password" and "no such account" for enumeration protection) with a UI that offers every real next step — create an account, try Google, or try Apple — rather than a dead end
- Detects and resolves the "this email already has a password account" conflict for OAuth sign-ins, offering to link the OAuth credential automatically
- Reserved, globally-unique `@usernames`, race-safe against two people claiming the same name simultaneously
- Real avatar photo upload (compressed client-side, stored inline — no Firebase Storage)

Invites & Sharing

- Invite by email (Google accounts)
- Invite by `@username` (works for any account type, including guests who've set one)

- Shareable invite link (`/join/<boardId>`) that anyone can open to join instantly
- "Join with an invite link" dialog for pasting a link/id directly inside the app

Permissions

- Real per-board membership — a board is invisible to anyone not on its member list (enforced by Firestore rules, not just hidden UI)
- Owner / Editor / Viewer roles; a Viewer can read a board but every write path (create task, edit, comment, drag, delete) is blocked server-side
- The four seeded demo boards are visible to guests only, so a first-time visitor sees a populated, editable sandbox instantly, while a real account starts with a genuinely empty board list — matching how a real multi-tenant app behaves

Everything Else

- Notifications (assignment, comments) with "load more" pagination
- Real search across title + description, with priority and assignee filters
- Activity log per board, aggregated across all its tasks
- Light/Dark/System theme
- Onboarding flow and a branded splash screen (HTML pre-load splash + in-app equivalent)

3. Technology Stack

LAYER	CHOICE	WHY
UI framework	Flutter (Dart)	Single codebase targeting Web (primary deployment) and Android; compiles to a real native-feeling app, not a wrapped webview.
State management	Riverpod (<code>flutter_riverpod</code>)	<code>StreamProvider</code> / <code>StateNotifierProvider.autoDispose</code> map cleanly onto Firestore's snapshot-listener model, and <code>.family</code> providers scope state per board/task/user id.
Backend	Firebase Firestore	Real-time listeners are the actual sync mechanism — no custom WebSocket server, no polling.
Auth	Firebase Authentication	Google, Apple, email/password, and anonymous providers, all first-class.
Hosting	Firebase Hosting	Static web build deployment with a single-page-app rewrite rule.
AI	Google Gemini API (<code>gemini-3.6-flash</code>)	Free-tier accessible via a plain REST call from the client, compile-time key injection.
Image handling	<code>image_picker</code> + <code>image</code> (pure Dart)	Client-side resize/compress/encode avoids needing Firebase Storage (which now requires the paid Blaze plan).

Key packages (pubspec.yaml)

```
flutter_riverpod: ^2.6.1      # state management
google_fonts: ^6.2.1
firebase_core: ^3.8.0
cloud_firestore: ^5.5.0
firebase_auth: ^5.3.3
http: ^1.2.2                 # Gemini REST calls
shared_preferences: ^2.3.3    # onboarding-seen flag
intl: ^0.19.0
uuid: ^4.5.1
image_picker: ^1.1.2         # avatar photo picking
image: ^4.2.0                 # client-side compression

dev:
fake_cloud_firestore: ^3.1.0  # provider unit tests without real Firebase
firebase_auth_mocks: ^0.14.1
flutter_lints: ^6.0.0
```

4. System Architecture

FlowBoard has no custom backend server. All persistence, real-time sync, and access control run through Firestore and its Security Rules; all compute (aside from the client itself) is the Gemini API called directly from the browser. This keeps the entire system free to run and easy to reason about, at the cost of pushing more logic (especially authorization) into declarative security rules rather than server code.

Layering

- **Screens** (`lib/screens/`) — one file per full-page route, mostly `ConsumerWidget` / `ConsumerStatefulWidget` .
- **Widgets** (`lib/widgets/`) — reusable pieces shared across screens (member avatar, task card, invite sheet, empty states, etc.).
- **Providers** (`lib/providers/`) — all Firestore/Auth interaction lives here. Screens never call `cloud_firestore` directly except through a provider or notifier method.
- **Models** (`lib/models/`) — plain immutable Dart classes with no Firestore knowledge of their own.
- **Data** (`lib/data/`) — mapping functions between Firestore documents and models (`firestore_mappers.dart`), the one-time demo-data seed, and static mock/persona data.
- **Services** (`lib/services/`) — the Gemini HTTP client and web-only deep-link/URL-strategy glue.

Real-time sync, concretely

A typical write path looks like: user drags a card → `BoardTasksNotifier.moveTask()` computes a new fractional order → one Firestore `update()` call → every open client (including the one that made the move) receives the change back through the same `snapshots()` listener that populates the board. There is no separate "optimistic local update" — what you see after dropping the card is the same round trip every other viewer sees, just fast enough (typically well under a second) to feel instant. This was a deliberate choice: it's slightly slower than optimistic UI, but it means the demo is not lying about being real-time.

5. Data Model (Firestore Schema)

Top-level collections

COLLECTION	DOCUMENT ID	PURPOSE
<code>boards</code>	auto or fixed (seed ids <code>b1</code> – <code>b4</code>)	One document per board.
<code>boards/{id}/tasks</code>	auto	Subcollection: one document per task card.
<code>boards/{id}/presence</code>	uid	Subcollection: live-viewer heartbeats.
<code>users</code>	uid	One document per account (profile, email, username, photo, bio).
<code>users/{uid}/notifications</code>	auto	Subcollection: per-user notification feed.
<code>usernames</code>	lowercase username	Reservation map: <code>{ uid }</code> , keyed by the name itself so uniqueness is a document-existence check.
<code>meta</code>	<code>seed</code>	One-time marker doc guarding the demo-data seed against double-seeding from concurrent app launches.

`boards/{boardId}` fields

FIELD	TYPE	NOTES
<code>name</code>	string	
<code>color</code>	int (ARGB32)	
<code>ownerId</code>	string (uid)	Always resolves to role <code>owner</code> regardless of the <code>roles</code> map.
<code>members</code>	array<map>	Denormalized display list: <code>{id, name, initials, color}</code> per member.
<code>memberIds</code>	array<string>	Kept in sync with <code>members</code> ; exists purely so security rules can do a cheap <code>uid in memberIds</code> check.
<code>roles</code>	map<uid, string>	<code>"editor"</code> or <code>"viewer"</code> ; missing entry defaults to editor.
<code>isDemo</code>	bool	True only on the four seeded boards; makes them guest-visible.
<code>linkJoinEnabled</code>	bool	Toggle controlling whether the shareable invite link currently admits new members.

<code>archived</code>	bool	Hides the board from the main list without deleting it.
<code>updatedAt</code>	timestamp	Server timestamp, bumped on every board-level write.

`boards/{id}/tasks/{taskId}` **fields**

FIELD	TYPE	NOTES
<code>title</code> , <code>description</code>	string	
<code>priority</code>	string	<code>low</code> / <code>medium</code> / <code>high</code>
<code>column</code>	string	<code>todo</code> / <code>inProgress</code> / <code>done</code>
<code>order</code>	double	Fractional index within the column — see §9.
<code>assignee</code>	map	Denormalized member snapshot.
<code>dueDate</code>	timestamp?	Optional.
<code>subtasks</code>	array<map>	<code>{id, text, done}</code> ; AI-generated or manual.
<code>comments</code>	array<map>	<code>{id, author, time, body}</code> .
<code>activity</code>	array<map>	Append-only timeline entries (created, moved, commented).
<code>updatedAt</code> / <code>updatedBy</code>	timestamp / map	Drives conflict-banner detection; every write must attribute itself truthfully (enforced by rules).

`users/{uid}` **fields**

FIELD	TYPE	NOTES
<code>id</code> , <code>name</code> , <code>initials</code> , <code>color</code>	—	Core <code>Member</code> shape, duplicated wherever a Member snapshot is embedded elsewhere.
<code>email</code>	string?	Lowercase-normalized; present only for non-anonymous accounts. Backfilled lazily on sign-in for accounts created before this field existed.
<code>username</code>	string?	Mirrors the key that owns this uid in the <code>usernames</code> collection.
<code>photoBase64</code>	string?	A compressed JPEG, base64-encoded, capped at ~180 KB. Watched live per-uid by every avatar in the app (see §9).
<code>bio</code>	string?	Free-text, capped at 160 characters client-side.

6. Authentication & Identity

Sign-in methods

- **Google** — `signInWithPopup(GoogleAuthProvider())`
- **Apple** — `signInWithPopup(OAuthProvider('apple.com'))` with `email` and `name` scopes requested
- **Email / password** — standard Firebase email-password auth, plus account creation and password reset
- **Guest** — `signInAnonymously()` ; guests are assigned a random persona (name/initials/color) so a handful of guest tabs look like distinct collaborators

The ambiguous-credential problem

Modern Firebase Auth deliberately returns the same `invalid-credential` error code for "wrong password" and "no account exists" (an email-enumeration protection). The sign-in screen can't tell these apart, so instead of guessing, it presents a dialog offering every real path forward: create a new account, try Google, or try Apple. If an OAuth attempt then reveals the email is already registered with a password (Firebase attaches the email to that specific exception), the email field is pre-filled and the app offers to link the OAuth credential to the existing account automatically once the correct password is entered — unifying what would otherwise become two disconnected accounts for the same person.

Profile resolution

`currentMemberProvider` (a `FutureProvider`) is the single place a signed-in Firebase `User` becomes a FlowBoard `Member`. On first sign-in it creates the `users/{uid}` document; on every sign-in it opportunistically backfills the `email` field for accounts created before that field existed. The resolved value is mirrored into `currentMemberStateProvider` (a plain `StateProvider`) so the rest of the app can read the current member synchronously instead of threading a `FutureProvider` through every widget.

7. Security Model & Permissions

Every access-control decision in FlowBoard is enforced in Firestore Security Rules, not just hidden in the UI. This section explains the model; the full rules listing is in Appendix A.

Membership

A board carries `memberIds` (an array of uids) and a `roles` map. Reading, writing, or even knowing a board exists requires `request.auth.uid in memberIds` — with two narrow, deliberate exceptions described below. A member with role `viewer` can read the board and its tasks but every write path is rejected server-side: creating a task, editing one, commenting, dragging, deleting. The board owner can always change the `roles` map; a plain editor cannot promote or demote anyone.

Exception 1 — demo boards, guests only

The four seeded boards carry `isDemo: true`. The rules grant read/write access to them for *anonymous* sign-ins only (checked via `request.auth.token.firebase.sign_in_provider == 'anonymous'`) — a real account never sees them. This preserves the "click the link, see a populated board instantly" demo experience for casual visitors while keeping real accounts on a genuinely private-by-default model.

Exception 2 — invite links

A board with `linkJoinEnabled: true` can be read by any signed-in user, and — critically — a non-member is allowed to *update* it, but the rule narrowly restricts what that update may contain: it must add only the requester's own uid to `memberIds`, assign only their own key in `roles` (and only to `"editor"`), and touch no other field. This was verified directly against the rules (not just the UI) for the failure cases: a join attempt that tries to self-promote to owner, add an unrelated uid, or sneak in a change to the board name in the same write is all rejected with `PERMISSION_DENIED`.

Username reservations

Uniqueness is enforced by Firestore's own write semantics rather than a transaction: the `usernames/{name}` document's `create` operation only succeeds if the document doesn't already exist. Two users racing to claim the same name can't both win — Firestore serializes the two writes, and the loser's request is evaluated as an `update` (which no rule permits), not a `create`. The client still pre-checks availability for a nicer error message, but the actual race-safety guarantee comes from the rule, not the client.

Identity integrity

Every task write must carry `updatedBy.id == request.auth.uid` — you cannot spoof another user as the author of a change. This is what makes the conflict-detection banners and activity log trustworthy: the UI can rely on "someone else changed this" being a claim the server actually verified, not just data the client happened to write.

8. Real-Time Collaboration Mechanics

Fractional-index drag-and-drop

Moving a card to a new position computes a new `order` value as the midpoint between its new neighbors' existing order values (or ± 1000 at either end of a column). This means reordering one card is always a single-document write — no sibling documents are touched, no batch rewrite of an entire column's indices is ever needed, and the approach scales to arbitrarily long columns without degrading.

Presence

Firestore has no native "this client disconnected" signal (unlike Realtime Database). Presence is therefore heartbeat-based: while a board screen is open, a `PresenceHeartbeat` writes a refreshed timestamp to `boards/{id}/presence/{uid}` every 12 seconds and deletes it on clean dispose. A companion `PresenceNotifier` re-filters the visible list every 8 seconds against a 25-second staleness cutoff, so a tab that closed uncleanly (crash, force-quit) still drops off the presence list within about half a minute instead of lingering forever.

Conflict detection

When a task detail sheet opens, it captures the task's current `updatedAt` as a baseline. If the live Firestore listener later reports a newer `updatedAt` attributed to a different `updatedBy` than the current viewer, a banner appears naming who made the change and when. This is a real comparison against live server data — not a simulated "someone else is typing" indicator.

9. Feature Deep Dives

Avatar photos

Firebase Storage now requires the paid Blaze plan even for free-tier usage, so photos are stored as a compressed JPEG (client-resized to a 256px-longest-side thumbnail, quality-stepped down until it fits under ~180 KB) encoded as base64 directly on the `users/{uid}` document. Rather than duplicating that data into every place a `Member` snapshot is embedded (task assignees, comments, presence, board member lists — which would be both wasteful and go stale), `MemberAvatar` is a live widget that watches `memberPhotoProvider(member.id)` — a per-uid Firestore listener — so a photo set on the profile screen appears everywhere that member is rendered, immediately, without any of those other documents changing at all.

Browser gesture note: the photo picker must be triggered with no `await` gap back to the original tap, or the browser refuses to honor the hidden file input's `click()` as a genuine user gesture and the native picker silently never opens. The camera button therefore calls the picker directly rather than routing through an intermediate options dialog (a separate `"..."` menu handles the secondary color/remove options instead).

Invite by link

Enabling a board's invite link sets `linkJoinEnabled: true`. The link itself is `https://flowboard-app-7539.web.app/join/{boardId}`, served via `usePathUrlStrategy()` for a clean URL with no `#`. The board id is captured from `Uri.base` as the very first statement in `main()` — before `WidgetsFlutterBinding` or anything else runs — because Flutter's own web navigation integration was found (through live testing) to silently normalize the address bar back to `/` shortly after startup, which would otherwise lose the pending join id before it could be read. Once the signed-in user's profile resolves, the app joins them to that board and navigates straight to it using an app-level `navigatorKey` / `scaffoldMessengerKey` rather than a widget's `BuildContext`, since the join is a network round trip that can outlive whichever screen happened to schedule it.

AI subtasks

Calls the Gemini API (`gemini-3.6-flash`, via a plain REST POST) with the task's title and description, asking for a short checklist. The API key is injected at compile time via `--dart-define=GEMINI_API_KEY=...` and read with `String.fromEnvironment` — never committed to source. If no key is configured, or the call fails, a static fallback suggestion list is used instead so the feature still demos.

Search

Matches on both title and description (case-insensitive substring), combined with optional priority and assignee filters — all applied client-side over the already-synced task list for a given board, since the dataset size per board makes a full Firestore query unnecessary.

10. UI/UX & Design System

Visual design was sourced from a Claude-generated design export (a `.dc.html` canvas export with JSX-like `sc-if` / `sc-for` templating) and PNG screenshots for individual screens, cross-referenced against each other for screens where both existed. Tokens were ported into `lib/theme/`:

FILE	CONTENTS
<code>app_colors.dart</code>	Full light/dark palette, priority colors, brand primary (<code>#5B45E0</code>).
<code>app_text_styles.dart</code>	Named text styles (h1/h2/h3, body variants, meta labels) matching the design system's type scale.
<code>app_metrics.dart</code>	Shared corner radii and spacing constants.
<code>app_theme.dart</code>	Assembles the above into Flutter <code>ThemeData</code> for light and dark mode.
<code>priority_colors.dart</code>	Per-priority accent/background/text color triples.

The brand mark itself (`FlowBoardLogo`) is drawn as vector shapes — a rounded-square background behind three vertical bars of different heights — rather than an image asset, so it scales cleanly at any size from the 20px header wordmark up to the 96px profile avatar corner badge, and stays trivially theme-aware.

12. Getting Started (Local Development)

Prerequisites

- Flutter SDK (Dart ^3.11.1 or newer)
- A Firebase project with Firestore and Authentication enabled
- The Firebase CLI (`npm i -g firebase-tools`) and `flutterfire_cli`
- (Optional) a Google Gemini API key for real AI subtasks — see aistudio.google.com/apikey

Setup

```
# Install dependencies
flutter pub get

# Generate lib/firebase_options.dart for your own Firebase project
flutterfire configure

# Enable sign-in providers you want in Firebase Console → Authentication:
#   Google, Apple (needs an Apple Developer "Services ID" configured
#   with a return URL of https://<project>.firebaseapp.com/__/auth/handler),
#   Email/Password, Anonymous

# Deploy security rules
firebase deploy --only firestore:rules

# Run locally (web)
flutter run -d chrome --dart-define=GEMINI_API_KEY=your_key_here
```

Running tests

```
flutter analyze
flutter test
```

13. Testing Strategy

Two complementary layers of testing were used throughout development:

Automated (unit & widget tests)

Provider logic is tested against `fake_cloud_firestore` and `firebase_auth_mocks` — a real (in-memory) Firestore API surface without needing a live project. This covers fractional-index math, conflict-attribution on moves, notification triggers, membership/role mutations, invite-link joins, and username-reservation race conditions. Widget tests cover key screens (splash, sign-in with all four auth options, the empty-state join-by-link dialog end to end, onboarding) using `ProviderScope` overrides. At the time of writing, **29 tests** pass across 5 test files.

Manual/live (Playwright)

Because so much of the app's value is in genuinely-real multi-user behavior, many features were verified against the live deployed app using Playwright browser automation — launching separate browser contexts to simulate truly distinct users (not just two tabs sharing a session), including real email/password test accounts for scenarios OAuth popups can't script. This caught several real defects that unit tests alone would have missed, notably:

- Firestore rejecting an entire unfiltered collection query once its rule depended on per-document fields — the boards list going silently empty for every user right after a rules deploy
- A stale build cache silently dropping a web plugin's registration (`MissingPluginException` on every photo-picker call)
- An avatar photo with transparent pixels rendering as a see-through hole instead of falling back to the member's color
- A "Join with an invite link" button that only ever showed a placeholder snackbar

14. Deployment

```
# Build the web bundle (release, with the AI key baked in)
flutter build web --release --dart-define=GEMINI_API_KEY=your_key_here

# Deploy hosting only
firebase deploy --only hosting

# Deploy security rules only (after any firestore.rules change)
firebase deploy --only firestore:rules
```

Migration note: when the membership/roles model was introduced, existing production board documents needed a one-time backfill (`memberIds` , `roles` , `isDemo` for the seed boards) run *before* the stricter rules were deployed — otherwise every board would have failed the new rule's `uid in memberIds` check immediately, since none of them had that field yet. Any future rules change that gates on a new field needs the same care: backfill under the old rules, verify, then deploy the new rules.

15. Known Limitations & Future Work

LIMITATION	REASON
No iOS build verified	Requires macOS + Xcode; not available in this project's development environment. The Android build is verified (<code>flutter build apk</code> succeeds).
Gemini API key has no website-restriction	Newer Gemini keys are service-account-bound, and Google does not currently offer HTTP-referrer restriction for that key type — only API-scope restriction, which is applied. A backend proxy would fully solve this but requires the paid Blaze plan.
Avatar photos are size-capped (~180 KB)	Stored inline on the user document rather than in Firebase Storage, to avoid the Blaze-plan requirement; this caps practical photo quality.
No push notifications	In-app notification feed only; browser/mobile push would need FCM wiring, not yet built.
Search/filter is client-side per board	Fine at demo scale; a board with a very large number of tasks would benefit from server-side query filtering instead.

16. Development History

High-level summary of the project's commit history, oldest to newest:

1. **Initial commit** — full Flutter + Firebase scaffold: real-time sync, presence, conflict detection, drag-and-drop, Google/email/guest auth, notifications, Gemini AI subtasks.
2. **Membership & roles** — per-board `memberIds` / `roles`, task deletion, archived-board recovery, anonymous-to-Google account linking.
3. **Hotfix** — boards list breaking under the new membership-gated rules (unfiltered collection query rejected outright); split into two provably-safe queries.
4. **Sign-in UX fixes** — resolved the ambiguous-credential dead end; added a profile avatar color picker.
5. **Guest-only demo boards** — restricted the four seed boards to anonymous sessions only, both client-side and via rules.
6. **Real invite links** — `linkJoinEnabled` toggle, narrowly-scoped self-join security rule, `/join/{id}` deep link handling.
7. **Avatar photos & usernames** — live per-uid photo watching, race-safe username reservation, invite-by-username.
8. **Splash screen, sign-in polish** — branded HTML + in-app splash, Google/Microsoft icons and field labels added to match a design reference.
9. **OAuth error-message accuracy fix** — surfaced during testing of the new provider button.
10. **"Join with an invite link" wired up** — replaced a leftover placeholder with a real paste-a-link dialog.
11. **Apple sign-in** — replaced Microsoft/Azure entirely with Apple, per final product direction; verified live against Firebase's real auth handler.

17. Appendix A — Full Firestore Security Rules

```
rules_version = '2';

service cloud.firestore {
  match /databases/{database}/documents {

    function isSignedIn() {
      return request.auth != null;
    }

    function isSelf(uid) {
      return request.auth.uid == uid;
    }

    function isAnonymousUser() {
      return isSignedIn() && request.auth.token.firebase.sign_in_provider == 'anonymous';
    }

    match /meta/{docId} {
      allow read, write: if isSignedIn();
    }

    match /users/{uid} {
      allow read: if isSignedIn();
      allow write: if isSignedIn() && isSelf(uid);

      match /notifications/{notifId} {
        allow read, update, delete: if isSignedIn() && isSelf(uid);
        allow create: if isSignedIn();
      }
    }

    match /usernames/{name} {
      allow read: if isSignedIn();
      allow create: if isSignedIn() && request.resource.data.uid == request.auth.uid;
      allow delete: if isSignedIn() && resource.data.uid == request.auth.uid;
      allow update: if false;
    }

    match /boards/{boardId} {
      function isDemoBoard() {
        return resource.data.isDemo == true && isAnonymousUser();
      }
      function isMember() {
        return isSignedIn() && request.auth.uid in resource.data.memberIds;
      }
      function isMemberOfNew() {
        return isSignedIn() && request.auth.uid in request.resource.data.memberIds;
      }
      function myRole() {
        return resource.data.roles[request.auth.uid];
      }
      function rolesUnchanged() {
```

```

    return request.resource.data.roles == resource.data.roles;
}
function isOwner() {
    return isSignedIn() && resource.data.ownerId == request.auth.uid;
}
function isLinkJoinEnabled() {
    return resource.data.linkJoinEnabled == true;
}
function isSelfJoiningViaLink() {
    return isLinkJoinEnabled()
        && !(request.auth.uid in resource.data.memberIds)
        && request.resource.data.memberIds == resource.data.memberIds.concat([request.auth.uid])
        &&
request.resource.data.roles.diff(resource.data.roles).affectedKeys().hasOnly([request.auth.uid])
        && request.resource.data.roles[request.auth.uid] == 'editor'
        && request.resource.data.diff(resource.data).affectedKeys().hasOnly(['members',
'memberIds', 'roles', 'updatedAt']);
}

allow read: if isSignedIn() && (isDemoBoard() || isMember() || isLinkJoinEnabled());
allow create: if isSignedIn()
    && isMemberOfNew()
    && request.resource.data.ownerId == request.auth.uid;
allow update: if isSignedIn()
    && (isDemoBoard()
        || (isMember() && myRole() != 'viewer' && (rolesUnchanged() || isOwner()))
        || isSelfJoiningViaLink());
allow delete: if isOwner();

match /presence/{uid} {
    allow read: if isSignedIn();
    allow write: if isSignedIn() && isSelf(uid);
}

match /tasks/{taskId} {
    function board() {
        return get(/databases/${database}/documents/boards/${boardId}).data;
    }
    function isDemoTaskBoard() {
        return board().isDemo == true && isAnonymousUser();
    }
    function isBoardMember() {
        return isSignedIn() && request.auth.uid in board().memberIds;
    }
    function isBoardViewer() {
        return board().roles[request.auth.uid] == 'viewer';
    }

    allow read: if isSignedIn() && (isDemoTaskBoard() || isBoardMember());
    allow create, update: if isSignedIn()
        && (isDemoTaskBoard() || (isBoardMember() && !isBoardViewer()))
        && request.resource.data.updatedBy.id == request.auth.uid;
    allow delete: if isSignedIn() && (isDemoTaskBoard() || (isBoardMember() &&
!isBoardViewer()));
}
}

```

```
}  
}
```

18. Appendix B — Configuration & Environment Variables

VARIABLE / FILE	PURPOSE
<code>--dart-define=GEMINI_API_KEY=...</code>	Compile-time constant read via <code>String.fromEnvironment</code> ; never committed to source. Without it, AI subtasks fall back to a static list.
<code>lib/firebase_options.dart</code>	Generated by <code>flutterfire configure</code> ; contains per-platform Firebase app config (API keys, project id, app ids). These are client identifiers, not secrets — Firestore/Auth security comes entirely from Security Rules, not from hiding this file.
<code>firestore.rules</code>	Deployed via <code>firebase deploy --only firestore:rules</code> . See Appendix A for the full listing.
Firebase Console → Authentication → Sign-in method	Google, Apple, Email/Password, and Anonymous must each be explicitly enabled (and saved) for their respective sign-in buttons to function.
Apple provider (Firebase Console)	Requires an Apple Developer "Services ID" with Sign in with Apple enabled and a return URL of <code>https://<project>.firebaseapp.com/__/auth/handler</code> , plus the generated key ID/team ID/private key entered into Firebase's Apple provider settings.

End of document. FlowBoard — a portfolio project built to demonstrate real-time collaborative systems engineering with Flutter and Firebase, without relying on any paid infrastructure.